

Tema: Productor consumidor

Presentado por: Jhon Andrioly Zenteno Sosa

2024-119066

Analisis de resultados.

1. **prodcons_01.c -> Provee una implementación del algoritmo presentado en la figura 5.9 del texto guía. ¿Qué problema ve usted en esta solución?**

El problema principal es una condición de carrera con pérdida de señales de sincronización, lo cual conduce a que el consumidor intente extraer elementos cuando el búfer está vacío, generando valores negativos en el contador.

```
jhon@jhon-PC:~/Escritorio/03_productorConsumidor$ gcc -o prodcons_01 prodcons_01.c -pthread
jhon@jhon-PC:~/Escritorio/03_productorConsumidor$ ./prodcons_01
[P] Producing
[P] Produced
[P]   Appending
[P]   Appended
[P]       item: 1
[P] Producing
[C] Taking
[C] Taked
[C]       item: 1
[C]   Consuming
[C]   Consumed
[P] Produced
[P]   Appending
[P]   Appended
[P]       item: 1
[P] Producing
[C] Taking
[C] Taked
[C]       item: 1
[C]   Consuming
[C]   Consumed
```

```

[P]          item: -1
[P] Producing
[C] Consumed
[C] Taking
[C] Taked
[C]          item: -1
[C] Consuming
[C] Consumed
[C] Taking
[P] Produced
[C] Taked
[C]          item: -2
[C] Consuming
[P] Appending
[C] Consumed

[P] Appended
[P]          item: -2
[P] Producing
[C] Taking
[C] Taked
[C]          item: -2
[C] Consuming
[P] Produced

```

2. **prodcons_01_buggy_01.c -> Esta implementación demuestra que un posible error de digitación en el código puede llevar a un comportamiento anormal en la ejecución del programa**

El error de digitación consiste en haber colocado `sem_post(&delay);` en lugar de `sem_wait(&delay);` al inicio de la función `consumer` (`// <--` a typo error). Como `delay` se inicializa en 0, el consumidor debía esperar a que el productor agregara el primer elemento. Sin embargo, al ejecutar `sem_post(&delay)`, el consumidor incrementa indebidamente el semáforo y entra directamente a extraer datos cuando el búfer aún está vacío.

```

jhon@jhon-PC:~/Escritorio/03_productorConsumidor$ gcc -o prodcons_01_buggy_01 prodcons_01_buggy_01.c -pthread
jhon@jhon-PC:~/Escritorio/03_productorConsumidor$ ./prodcons_01_buggy_01
[P] Producing
[C] Taking
[P] Produced
[C] Taked
[C] item: 0
[C] Consuming
[C] Consumed
[C] Taking
[C] Taked
[C] item: -1
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: -1
[P] Producing

```

3. **prodcons_01_buggy_02.c** -> Esta implementación muestra otro tipo de error de codificación y es cuando el programador altera el orden donde se ubican los semáforos

El cambio en el orden de los semáforos provoca un bloqueo mutuo si el consumidor se ejecuta primero.

```

jhon@jhon-PC:~/Escritorio/03_productorConsumidor$ ./prodcons_01_buggy_02
[P] Producing
[P] Produced

```

Al iniciar, el consumidor adquiere primero el semáforo de exclusión mutua `s` y lo deja en cero, inmediatamente después, el consumidor intenta ejecutar la espera sobre el semáforo `delay`, pero como este inicia en cero, el consumidor queda suspendido, por lo que cuando el productor intenta ingresar a su sección crítica mediante el semáforo `s`, queda bloqueado indefinidamente porque el consumidor nunca liberó dicho semáforo antes de dormirse. Como consecuencia, ambos procesos quedan esperando mutuamente por un recurso retenido y el programa se congela por completo

4. **prodcons_01_OK.c** -> Esta es una implementación correcta del problema de los productores consumidores de acuerdo a como lo sugiere la figura 5.10

Esta implementación resuelve la condición de carrera utilizando una variable local privada *m* en el hilo consumidor.

```
[P]           item: 1
[P] Producing
[C] Taking
[C] Taked
[C]           item: 1
[C] Consuming
[C] Consumed
[P] Produced
[P] Appending
[P] Appended
[P]           item: 1
[P] Producing
[P] Produced
[P] Appending
[P] Appended
[P]           item: 2
[P] Producing
[C] Taking
[C] Taked
[C]           item: 2
[C] Consuming
[P] Produced
[P] Appending
```

Al copiar $m = n$ dentro de la sección crítica antes de liberar el semáforo *s*, la decisión se toma con una copia privada que el productor no puede alterar fuera de tiempo, esto evita que el consumidor intente extraer de un búfer vacío y previene valores negativos en el contador.